
Offline Cryptography CTF Assignment System

Information Security

Sections 7D / 9D

Assignment type	Individual, personalized, offline browser-based investigation
Total marks	100
Due date	_____

Contents

1	Assignment Overview	2
2	Personalization and Fairness	2
3	What the Student Requires	2
3.1	Files received	2
4	Student Procedure	3
5	Required Technical Work	3
5.1	Monoalphabetic substitution program	3
5.2	Vigenere program	4
5.3	Rail Fence program	4
5.4	Final XOR program	4
6	Required Submission	4
6.1	Flags file	5
6.2	Report requirements	5
7	Theoretical Questions	5
8	Allowed and Prohibited Activities	6
8.1	Allowed	6
8.2	Prohibited	6
9	Marking Rubric	6
10	Submission and Verification Process	6
10.1	Student submission process	6

1 Assignment Overview

Students will receive a personalized ZIP archive containing a small fictional cybersecurity investigation portal. The portal is made entirely from local HTML, CSS, JavaScript, JSON, text, and supporting files. After downloading and extracting the archive, the student opens `index.html` in a normal browser. No web server, external website, API, CDN, or Internet connection is required after download.

The investigation contains three independently solvable cryptographic branches:

1. monoalphabetic substitution cryptanalysis;
2. Vigenere cipher decryption;
3. Rail Fence cipher decryption and depth testing.

Each completed branch reveals a student-specific stage flag and one ordered hexadecimal fragment. The three fragments are reconstructed in their stated order and used as the byte key for the final one-time-pad exercise. The final decryption produces a success message and a student-specific final flag.

AES and DES are assessed only through short theoretical questions. Students are not required to implement AES or DES.

2 Personalization and Fairness

Every student receives a different assignment instance generated from their normalized roll number and an instructor-only master secret. Depending on the generated instance, the following items differ between students:

- challenge texts and ciphertexts;
- substitution alphabet and the provided analysis information;
- Vigenere key derivation and ciphertext;
- Rail Fence message and actual depth;
- investigation route and placement of discoverable information;
- all stage flags;
- final XOR key fragments, ciphertext, and final flag; and
- local final-flag validation hash.

3 What the Student Requires

3.1 Files received

Each student receives exactly one ZIP archive named using their roll number, for example:

22L-7776.zip

The student must confirm that the roll number in the filename and inside the assignment matches their own roll number. A student who receives the wrong ZIP must not begin solving it and must contact the instructor.

4 Student Procedure

Students must follow this sequence.

1. Download the ZIP archive intended for their own roll number.
2. Check that the filename and displayed case roll number match their roll number.
3. Extract the entire ZIP into a normal local folder. Do not work directly inside the compressed archive.
4. Keep an unchanged copy of the original extracted evidence.
5. Open `index.html` in a modern browser.
6. Read the case instructions and inspect the supplied offline materials using normal browser and file-inspection techniques.
7. Create separate Python programs for the substitution, Vigenere, Rail Fence, and final XOR stages.
8. Record analysis output and reasoning while working; this evidence is required in the report.
9. Recover the three stage flags and the three ordered hexadecimal fragments.
10. Reconstruct the final byte key in the specified order and validate its length before applying XOR.
11. Recover the final message and final flag.
12. Optionally enter the final flag into the local vault page for an immediate experience-only check.
13. Complete `flags.txt`, prepare the PDF report, and verify all required deliverables.
14. Compress the submission folder and upload it before the stated due date.

Important: The offline vault does not submit or transmit an answer to the instructor. It performs only a local hash comparison. Official grading uses the submitted `flags.txt` and the instructor's private answer records.

5 Required Technical Work

5.1 Monoalphabetic substitution program

The submitted program must:

- read the supplied ciphertext;
- calculate and print a letter-frequency table;
- support a partial ciphertext-to-plaintext substitution mapping;
- preserve nonalphabetic characters appropriately; and
- produce the student's recovered plaintext when run with the student's completed mapping.

The report must explain how frequency information, word structures, supplied case information, and consistency checks contributed to the recovery.

5.2 Vigenere program

The submitted program must:

- implement the conventions stated in the student's case;
- derive and display the student's calculated key;
- preserve spaces and punctuation correctly;
- advance the key only as stated in the case; and
- decrypt the complete supplied ciphertext.

5.3 Rail Fence program

The submitted program must:

- implement the standard down-and-up zigzag Rail Fence algorithm;
- print candidate plaintexts for every depth from 2 through 8;
- clearly label every candidate depth; and
- identify the meaningful result through evidence rather than hard-coding a known answer.

5.4 Final XOR program

The submitted program must:

- accept the three hexadecimal fragments in their stated order;
- validate hexadecimal formatting;
- reconstruct the complete key as bytes;
- validate that key and ciphertext byte lengths are equal;
- XOR corresponding bytes; and
- decode and print the final recovered message.

Students must not describe Base64 as encryption. The final stage is byte-wise XOR, not AES or DES.

6 Required Submission

Students submit one ZIP named using the following convention:

```
ROLL-NUMBER_IS_CTF_Submission.zip
```

The archive must contain:

```
submission/  
|-- mono_solver.py  
|-- vigenere_solver.py  
|-- rail_solver.py  
|-- otp_solver.py  
|-- flags.txt  
'-- report.pdf
```

Do not submit virtual environments, cache folders, installed packages, unrelated screenshots, or a copy of the original assignment ZIP.

6.1 Flags file

The UTF-8 `flags.txt` file must use this exact structure:

```
ROLL_NUMBER=YOUR-ROLL-NUMBER
MONO_FLAG=YOUR-RECOVERED-MONO-FLAG
VIGENERE_FLAG=YOUR-RECOVERED-VIGENERE-FLAG
RAIL_FLAG=YOUR-RECOVERED-RAIL-FLAG
FINAL_FLAG=YOUR-RECOVERED-FINAL-FLAG
```

Students must replace every descriptive value with their own result. Flags are case-sensitive. Extra commentary must not be added to flag-value lines.

6.2 Report requirements

The PDF report should be concise, readable, and organized using the following headings:

1. Student name, roll number, section, and assignment title.
2. Investigation process and locations of relevant discovered information.
3. Monoalphabetic frequency table, partial mappings, major reasoning steps, and verification.
4. Vigenere key calculation and decryption procedure.
5. Rail Fence candidates for depths 2 through 8 and justification for the selected candidate.
6. Ordered fragment reconstruction, byte lengths, XOR procedure, and recovered final message.
7. Answers to the theoretical questions in Section 7.
8. Brief conclusion describing what was learned.

Screenshots may support the explanation, but screenshots alone are not a substitute for reasoning, source code, or text.

7 Theoretical Questions

Answer each question concisely in the report. AES and DES are theory-only topics in this assignment.

1. What block size and key sizes are standardized for AES, and why is AES preferred to DES for modern systems?
2. Why is the effective key size of DES inadequate against modern brute-force attacks?
3. What do confusion and diffusion mean in the design of a secure block cipher?
4. Why must a one-time-pad key be at least as long as its plaintext and used only once?
5. If the same one-time-pad key encrypts two messages, what relationship can an attacker calculate from the two ciphertexts, and why is this dangerous?

No AES or DES implementation is required or accepted as a substitute for these explanations.

8 Allowed and Prohibited Activities

8.1 Allowed

- Python 3 and its standard library.
- A text editor or IDE.
- Browser source/developer tools applied only to the supplied local assignment.
- Course slides, textbooks, and general references about cipher algorithms.
- General discussion of how an algorithm works, without sharing personalized data or code.

8.2 Prohibited

- Sharing or copying personalized ciphertexts, plaintexts, keys, fragments, flags, reports, or source code.
- Submitting another student's ZIP, answers, programs, or report content.
- Using an online cipher-solving or answer-generation service in place of the required implementation.
- Attacking institutional systems, accounts, networks, email, Drive permissions, or LMS infrastructure.
- Modifying client-side validation and presenting the visual result as evidence of a correct final flag.
- Implementing AES or DES instead of completing the required classical-cipher and XOR work.

All submitted code and report writing must be the student's own work. Similar or copied student-specific material may be investigated under the institution's academic-integrity policy.

9 Marking Rubric

Assessment Area	Marks
Website and evidence investigation	5
Monoalphabetic analysis and solver	10
Vigenere implementation and recovery	10
Rail Fence brute-force implementation	10
Final key reconstruction and XOR decryption	10
Correct student-specific flags	10
Individual code evaluation and viva	25
Technical report and theoretical questions	20
Total	100

10 Submission and Verification Process

10.1 Student submission process

1. Run all four programs from a clean folder and confirm they execute without missing local files.
2. Confirm that `flags.txt` contains the correct roll number and four nonempty flags.

3. Open `report.pdf` and confirm every page is readable.
4. Confirm that the submission archive contains exactly the required deliverables.

Program Execution Requirement

Each submitted Python program must be self-contained and executable from the submission folder using one command:

```
python mono_solver.py
python vigenere_solver.py
python rail_solver.py
python otp_solver.py
```

The instructor must not need to edit the program, provide command-line arguments, copy additional files, or enter values interactively before running it.

Students may include their student-specific ciphertext, completed substitution mapping, recovered key fragments, and other required parameters as clearly named constants inside the appropriate program.

However, the recovered plaintext must **not** be hardcoded or printed as a fixed answer. Each program must calculate the plaintext using the implemented cryptographic algorithm.

For example, the following is not acceptable:

```
print("This is the recovered plaintext...")
```

Instead, the program must decrypt the ciphertext and then print the calculated result:

```
plaintext = decrypt(ciphertext, recovered_parameters)
print(plaintext)
```

Each program should use the following structure:

```
def main():
    # Perform the required calculations
    # Decrypt the student-specific ciphertext
    # Print the required analysis and plaintext
if __name__ == "__main__":
    main()
```

When executed once, each program must:

- finish without an unhandled error;
- return exit status zero;
- print clearly labelled output;
- perform the required cryptographic calculation;
- print the calculated recovered plaintext;
- print the recovered stage flag where applicable; and

- produce the same result on repeated execution.

Programs that only print hardcoded plaintext or flag values without performing the required calculation will not receive implementation marks, even if the printed answers are correct.

Important Submission Instructions

Please follow all assignment and submission instructions carefully. Your work may be checked using automated Python tests. Therefore, incorrect filenames, missing files, incorrect folder structure, or an incorrectly formatted `flags.txt` file may cause the automated checks to report a failure even if part of your solution is correct.

Your submission ZIP must contain the following files inside one `submission/` folder:

1. `mono_solver.py`
2. `vigenere_solver.py`
3. `rail_solver.py`
4. `otp_solver.py`
5. `flags.txt`
6. `report.pdf`

The `flags.txt` file is especially important. It must be saved as a plain UTF-8 text file and must contain the following fields in the exact order shown:

```
ROLL_NUMBER=YOUR-ROLL-NUMBER
MONO_FLAG=YOUR-MONO-FLAG
VIGENERE_FLAG=YOUR-VIGENERE-FLAG
RAIL_FLAG=YOUR-RAIL-FLAG
FINAL_FLAG=YOUR-FINAL-FLAG
```

Replace every descriptive value with your own recovered result. Do not add quotation marks, spaces around the equals sign, additional comments, or renamed fields. Flags are case-sensitive.

Before submitting, run all four Python programs from the submitted folder and confirm that:

- every program executes without an error;
- all required input files can be found;
- hexadecimal input and key lengths are validated;
- `flags.txt` contains your own roll number and all four recovered flags;
- `report.pdf` opens correctly; and
- the final ZIP contains no virtual environment, cache folder, or unrelated files.

If the required files are missing, renamed, misplaced, corrupted, or submitted in the wrong format, the automated checks may produce an incorrect or failed result.

If you have any questions or are unsure about the submission format, please contact me before the deadline at:

1227776@lhr.nu.edu.pk :-)